

SHINOBI BATTLE

Project Report Document

A Computer Vision-Driven P2P Turn-Based Battle Game

AI / Computer Vision Final Project | Version 1.1

Table of Contents

Table of Contents.....	2
1. Project Overview	4
1.1 Academic Context	4
1.2 Project Goals	4
1.3 Out of Scope	4
2. Hand Sign System	5
2.1 Sign Vocabulary	5
2.2 Sequence Buffer.....	5
2.3 Jutsu Lookup Engine	6
2.4 Focus Action.....	6
3. Game Mechanics	7
3.1 Player Stats & Passive Regeneration	7
3.2 The Five Action Types	7
3.3 Jutsu Stats Schema.....	8
3.4 Failed Jutsu Chakra Penalty	8
3.5 Spam Dampening (Consecutive Use Penalty)	9
3.6 Shadow Clone (Dodge) Mechanics.....	9
3.7 Buff / Debuff System.....	9
3.8 Full Turn Resolution Flow	10
4. Sample Jutsu Registry	12
5. Computer Vision & AI System	14
5.1 Architecture Philosophy	14
5.2 Dataset Collection (Developer Pre-Game Step).....	14
5.3 Pipeline Stages (Runtime)	14
5.4 CNN Architecture.....	15
5.5 Optional Extension: CNN-LSTM Temporal Model.....	16
5.6 Data Augmentation (Applied Offline During Training).....	16
5.7 In-Game Calibration Screen	16
6. UI / UX Specification	17
6.1 Frontend Technology Stack.....	17
6.2 Naruto Theme Design Language	17
6.3 Battle Screen Layout	18
6.4 Action Banner Variants	18
6.5 Screen Navigation Flow	19
7. Networking Architecture	20

7.1 Technology Stack	20
7.2 Session Establishment	20
7.3 Message Protocol	20
8. Full Technical Stack	22
9. Development Phases & Milestones	23
10. Risks & Mitigations	24
11. Open Questions & Decisions Pending	Error! Bookmark not defined.
Appendix A - Glossary	25

1. Project Overview

Shinobi Battle is a two-player, turn-based battle game inspired by the Naruto anime series and the retro Pokemon game format. Its defining innovation is that player actions are not selected from a menu; they are performed physically. Each player performs a real-world sequence of two-handed signs (mirroring the hand seals from Naruto) in front of their webcam. A computer vision pipeline recognizes the combination and maps it to an in-game jutsu or action.

The game runs peer-to-peer over a cloudflare tunnel, allowing two players on separate laptops anywhere to battle each other with zero dedicated server infrastructure. The frontend UI runs in a browser tab, built with React and Vite, communicating with a Python WebSocket backend that hosts all authoritative game logic.

1.1 Academic Context

This project is the final submission for Computer Vision course. The CV component is the academic core: a Convolutional Neural Network (CNN) trained from scratch to classify hand sign skeleton images derived from MediaPipe landmark data.

1.2 Project Goals

- Collect a custom two-handed Naruto hand sign dataset (15 classes: 11 jutsu signs + Activation + Scrap + Shadow clone + Unknown) via a dedicated pre-game collection script run by the developer
- Train a CNN classifier on rendered skeleton images, targeting $\geq 90\%$ validation accuracy
- Implement a full turn-based battle engine with passive turn-start chakra regeneration, chakra mechanics, buff/debuff state, and spam dampening
- Build a peer-to-peer multiplayer session over ngrok with authoritative server-side game state
- Deliver a polished, Naruto-themed React UI with split-screen camera feeds, HP/chakra bars, private sign buffers, and animated action banners

1.3 Out of Scope

- Multiple playable characters or character roster
- Persistent accounts, ranked ladders, or match history
- Mobile deployment

2. Hand Sign System

2.1 Sign Vocabulary

All hand signs are two-handed, performed with both hands simultaneously, mirroring how jutsu hand seals are performed in the Naruto series. The Ram sign is designated as the Activation token and is therefore removed from the playable jutsu sign pool. The total CNN vocabulary is 15 classes:

#	Sign Name	Role in Game
1	Rat	Jutsu sign
2	Ox	Jutsu sign
3	Tiger	Jutsu sign
4	Hare	Jutsu sign
5	Dragon	Jutsu sign
6	Horse	Jutsu sign
7	Snake	Jutsu sign
8	Monkey	Jutsu sign
9	Bird	Jutsu sign
10	Dog	Jutsu sign
11	Boar	Jutsu sign
12	Ram	RESERVED - Activation token: finalizes the current sequence and submits it to the game engine
13	Scrap	RESERVED - instantly clears the sequence buffer without ending the turn
14	Shadow Clone	RESERVED - Unique two-handed gesture for Shadow Clone activation
15	Unknown	CNN training class only: absorbs ambiguous poses and transitions between signs

Ram is chosen as the Activation sign because it is one of the most visually distinct and widely recognized Naruto hand signs, making accidental triggering unlikely. The Scrap sign uses a crossed-wrists pose specifically because it is maximally distinct from all 11 jutsu signs and from Ram. The Unknown class is a CNN training class only and has no in-game action.

2.2 Sequence Buffer

- When a player's turn begins, their sequence buffer is empty
- Each confirmed sign from the 11 jutsu signs is appended to the ordered buffer in real time, with a brief lock-in animation and confidence flash in the private HUD

- The buffer contents are displayed privately in the caster's own HUD panel; the opponent cannot see them
- Scrap Sign: instantly empties the buffer; the player can begin signing again without ending their turn
- Activation Sign (Ram): finalizes the buffer and submits it to the Jutsu Lookup Engine; turn resolution begins
- Maximum buffer length is 6 signs (excluding Activation); once full, additional confirmed signs are rejected until the player Activates or Scraps, and the UI shows a buffer-full warning
- Shadow Clone Gesture: This gesture does NOT append to the buffer. When detected, it immediately triggers the Shadow Clone action, bypassing the sequence buffer and Activation sign.

2.3 Jutsu Lookup Engine

On Activation, the sequence buffer is checked against the Jutsu Registry. Three outcomes are possible:

1. Match Found: The corresponding jutsu fires. All stat math is applied and the action banner is shown above the caster's panel.
2. No Match: The sequence does not correspond to any registered jutsu. The player loses chakra proportional to the number of signs used (see Section 4.3). Turn ends.
3. Insufficient Chakra: Even with a match, if the caster cannot cover the jutsu's `chakra_cost_self`, the jutsu fails with a not-enough-chakra banner. Turn ends with no chakra refund.

2.4 Focus Action

If the player performs the Activation Sign (Ram) immediately with an empty buffer (no preceding signs), the action is interpreted as Focus: a deliberate chakra restoration action. Combined with the passive turn-start chakra grant (Section 3.1), a Focus turn yields a total of +30 chakra for that turn. Focus always succeeds regardless of current chakra level.

3. Game Mechanics

3.1 Player Stats & Passive Regeneration

Stat	Default Value	Description
Max HP	100	Hit points. Reaching 0 = loss.
Max Chakra	100	Chakra resource. Starts full each match.
Passive Turn Regen	+10	Chakra granted automatically at the START of every turn, before the player acts. Cannot exceed Max Chakra.
Focus Restore	+20	Additional chakra granted when the player performs a Focus action. Stacks with Passive Turn Regen for a total of +30 that turn.
Base Miss Immunity	0%	Inherent miss reduction (modifiable by buffs).

The passive +10 chakra at turn start ensures players always have some chakra coming in, making the game about smart jutsu selection rather than chakra starvation. Focus remains a meaningful action because it doubles the regeneration rate for that turn, at the cost of spending the turn doing nothing offensive or defensive.

Example: A player starts their turn with 30 chakra. They receive the passive +10, bringing them to 40. They then choose Focus, receiving another +20, ending the turn at 60 chakra.

3.2 The Five Action Types

Action	How Triggered	Core Effect	Announcement Banner
Attack Jutsu	Sign sequence + Ram (Activation)	Deals HP damage to opponent; optionally drains their chakra	e.g. FIREBALL JUTSU!
Buff Jutsu	Sign sequence + Ram (Activation)	Applies a positive stat modifier to self for X turns	e.g. POWER SURGE! (Buff Active)
Debuff Jutsu	Sign sequence + Ram (Activation)	Applies a negative stat modifier to opponent for X turns	e.g. SEAL DISRUPTION! (Debuff)
Shadow Clone	Unique gesture (no Activation required)	Costs 50% current chakra; opponent's next attack gets +50% miss	SHADOW CLONE JUTSU!
Focus	Ram (Activation) only with empty buffer	Grants +20 chakra on top of passive +10 regen. Total: +30.	Focus...

The action banner appears as a styled overlay at the top of the caster's camera panel for 2.5 seconds after the action resolves, visible to both players. The same HUD area also gives sign-confirmation feedback and lock-in cues when a gesture is recognised.

3.3 Jutsu Stats Schema

Field	Type	Description
name	string	Display name shown in the action banner
sign_sequence	list[str]	Ordered list of sign names (from the 11 jutsu signs) required to trigger
type	enum	attack buff debuff shadow_clone
dmg_to_enemy	int	HP damage dealt to opponent on a successful non-missed cast
dmg_to_self	int	Recoil HP damage dealt to the caster (0 if none)
chakra_cost_self	int	Chakra consumed from the caster on cast
chakra_cost_enemy	int	Chakra drained from the opponent on a successful cast
miss_rate	float 0-1	Probability the jutsu produces no effect (chakra still consumed by caster)
buff_effect	dict null	Stat modifier payload (see Section 3.7 for modifier channels)
buff_duration	int	Number of turns the buff/debuff lasts; 0 = current turn only
spam_count	int runtime	Tracks consecutive uses for spam dampening; resets on any different action. Each jutsu also has a separate cooldown_turns value in config.

3.4 Failed Jutsu Chakra Penalty

When a sign sequence does not match any registered jutsu, the caster loses chakra calculated as:

$$\text{chakra_loss} = \text{BASE_PENALTY} + (\text{len}(\text{sequence}) \times \text{PER_SIGN_PENALTY})$$

Recommended defaults: BASE_PENALTY = 5, PER_SIGN_PENALTY = 3.

Signs Used	Chakra Lost	Design Intent
1 sign	$5 + 3 = 8$	Small penalty for an accidental single-sign mis-Activation
2 signs	$5 + 6 = 11$	Moderate penalty

Signs Used	Chakra Lost	Design Intent
4 signs	$5 + 12 = 17$	Significant; a 4-sign jutsu would be powerful if correct
6 signs	$5 + 18 = 23$	Heavy; the longest attempts carry the largest failure cost

3.5 Spam Dampening (Consecutive Use Penalty)

If a player uses the exact same Attack jutsu on consecutive turns, the effective `dmg_to_enemy` is reduced by a multiplier. Chakra costs and all other effects remain unchanged.

Consecutive Uses of the Same Attack Jutsu	Damage Multiplier
1st use (or after any break)	100% - full damage
2nd consecutive	70%
3rd consecutive	40%
4th+ consecutive	20% (hard floor)

- The counter resets when the player uses any different action (different jutsu, Focus, Shadow Clone, Buff, or Debuff)
- Missed attacks still increment the consecutive counter
- Buff, Debuff, Shadow Clone, and Focus actions are fully exempt from spam dampening

3.6 Shadow Clone (Dodge) Mechanics

- Triggered instantly when the Shadow Clone gesture is detected
- Does not require Activation (Ram)
- Ignores sequence buffer entirely
- Costs exactly 50% of the caster's current chakra after the passive +20 turn regen has already been applied, rounded down
- Minimum cost enforced: the caster can never be reduced below 1 chakra by this action alone
- Effect: the opponent's very next Attack jutsu is resolved with an additional +50% miss rate, stacked additively on top of that jutsu's own base `miss_rate`
- The dodge buff is consumed on the opponent's next attack, whether that attack hits or misses
- Shadow Clone cannot stack; casting it a second time before the opponent attacks replaces the buff and costs another 50%
- Shadow Clone is not subject to spam dampening

3.7 Buff / Debuff System

Buff and debuff jutsu inject modifier entries into the target player's active_modifiers list. All active modifiers are evaluated at action resolution time. Available modifier channels:

Channel Key	Applies To	Direction	Example
atk_damage_flat	Outgoing dmg_to_enemy on all attacks	Self buff	+8 flat damage on all attacks for 2 turns
incoming_damage_mult	Incoming dmg_to_enemy received	Self buff	0.7x incoming damage (Stone Skin) for 1 turn
chakra_cost_mult	Self chakra_cost_self on all jutsu	Self buff	0.8x chakra cost (Chakra Flow) for 3 turns
enemy_chakra_drain_flat	Outgoing chakra_cost_enemy on attacks	Debuff on opp	+10 opponent chakra drain per attack for 2 turns
focus_regen_mult	Chakra restored by opponent's Focus action	Debuff on opp	0.5x opponent Focus restore for 2 turns (passive regen unaffected)
miss_rate_add	Additive miss chance on opponent's attacks	Debuff on opp	+0.30 miss rate on all opponent attacks for 1 turn
turn_regen_mult	Passive +20 turn-start chakra of opponent	Debuff on opp	0.5x opponent passive regen for 2 turns

- Multiple modifiers can be active simultaneously on the same player
- Application order: flat additions first, then multipliers
- All buff durations tick down by 1 at the start of the affected player's turn, before regen and before any action
- Expired modifiers are removed before that turn's passive regen and action are processed

3.8 Full Turn Resolution Flow

4. Active modifiers on the current player tick down; expired ones are removed.
5. Passive +10 chakra regen is applied (modified by turn_regen_mult if a debuff is active). Chakra is capped at Max Chakra.
6. Player performs two-handed signs; each confirmed sign appends to the private buffer shown in their HUD.
7. Player performs Ram (Activation) to submit, or Scrap to clear and restart without ending the turn.
8. Jutsu Lookup Engine checks the buffer.
9. No match: chakra penalty applied (Section 3.4); turn ends.
10. Match: check if caster has enough chakra_cost_self. If not, jutsu fails with a not-enough-chakra banner; turn ends.
11. Miss roll: random float compared against (jutsu.miss_rate + active additive miss modifiers). If miss, banner shows MISSED; turn ends with chakra consumed.

12. Spam dampening multiplier applied to `dmg_to_enemy`.
13. All stat changes applied simultaneously: HP to both parties, chakra to both parties, buff/debuff entries injected.
14. Action announcement banner displayed above caster's panel for 2.5 seconds, visible to both players.
15. Win condition checked: if either player HP ≤ 0 , match ends.
16. Turn passes to opponent.

4. Sample Jutsu Registry

Sign sequence abbreviations: Tiger=T, Snake=Sn, Ox=Ox, Dog=D, Boar=Bo, Horse=H, Bird=Bi, Monkey=M, Rat=Rt, Dragon=Dr, Hare=Ha. Ram is NOT used in any sequence (it is the Activation sign). All values are starting suggestions; tune during playtesting.

Jutsu Name	Type	Sequence	Dmg Enem	Dmg Self	Chkr Self	Chkr Enem	Miss%	Buff / Notes
Fireball Jutsu	Attack	Sn-M-Bo-H-T	30	0	20	0	10%	Classic 5-sign fire attack
Fire Stream	Attack	Sn-H-T	15	0	10	0	5%	Quick, low cost
Chidori	Attack	Ox-Ha-M-Bo-H-Bi	38	5	25	0	15%	5 recoil damage to self
Shadow Shuriken	Attack	T-Bo	10	0	8	0	25%	Cheap but unreliable
Wind Scythe	Attack	Bo-Sn-T-H	22	0	15	0	10%	Balanced mid attack
Chakra Drain Seal	Attack	Rt-Ox-M-Ha	5	0	15	20	10%	Drains 20 enemy chakra
Water Dragon	Attack	Ox-M-Ha-Bo-Sn-T-D-Bi	40	0	28	0	20%	High dmg, long 8-sign sequence
Stone Skin	Buff	Bo-D-Ha	0	0	12	0	0%	incoming_damage_mult 0.6 for 2 turns
Chakra Flow	Buff	Sn-Bi-H	0	0	10	0	0%	chakra_cost_mult 0.7 for 3 turns
Power Surge	Buff	T-M-Dr	0	0	15	0	0%	atk_damage_flat +8 for 2 turns
Seal Disruption	Debuff	Rt-Dr-Sn-Ox	0	0	18	0	5%	focus_regen_mult 0.4 on opponent for 3 turns
Smoke Blind	Debuff	D-Bo-H	0	0	12	0	0%	miss_rate_add +0.30 on opponent for 2 turns
Chakra Suppression	Debuff	M-Bi-T-D	0	0	20	0	10%	enemy_chakra_drain_flat +15 on opponent for 2 turns
Regen Seal	Debuff	Ox-Ha-D	0	0	14	0	0%	turn_regen_mult 0.5 on opponent for 3 turns (halves passive regen)
Shadow Clone	Dodge	[Reserved sequence]	0	0	50% current	0	0%	Next opp attack +50% miss
Focus	Focus	[Empty buffer + Ram]	0	0	0	0	0%	+20 chakra (stacks with passive +20 = +40 total this turn)

Sequences must be unique. The game validates the registry at startup and raises a configuration error if any two jutsu share the same `sign_sequence` tuple. Note that no sequence contains Ram, as Ram is permanently reserved as the Activation sign. Each jutsu also carries a `cooldown_turns` value in config, even though that field is not shown in this sample table.

5. Computer Vision & AI System

5.1 Architecture Philosophy

The CV pipeline combines MediaPipe Hands for landmark detection with a student-trained CNN for sign classification. The division of responsibility is deliberate and academically defensible:

- MediaPipe handles detection only: it finds the hands in the frame and returns 42 landmark points (21 per hand). This is standard industry tooling and is not the student's trained model.
- The CNN is the student's academic contribution. It receives a rendered skeleton image and classifies it into one of 15 sign classes. This is a genuine convolutional image classifier.
- Rendering to a skeleton image before CNN inference abstracts away skin tone, lighting, and background, dramatically reducing dataset size requirements while keeping the task genuinely visual (not tabular).

5.2 Dataset Collection (Developer Pre-Game Step)

The dataset is collected by the developer before the game is tested or deployed. It is not a step players perform at runtime. A dedicated `data_collector.py` script is provided for this purpose.

Parameter	Recommended Value	Notes
Samples per class	500-800	Collect from multiple people for generalization across hand sizes and skin tones
Total classes	15	11 jutsu signs + Ram (Activation) + Scrap + Unknown + Shadow clone
Total samples target	~8,000-11,000	Unknown class should include random poses
Lighting conditions	3+ settings	Bright daylight, indoor lamp, dim room
Background variety	2+ settings	Plain wall and cluttered desk at minimum
Distance range	40-80 cm	Vary per session to build distance invariance

The `data_collector.py` script shows a live webcam feed with a MediaPipe overlay. The developer presses a key to start recording samples for each sign class, naming each session by sign name. Collected landmarks are serialized to disk; the rendering to skeleton images happens offline during the training pipeline, not at collection time.

5.3 Pipeline Stages (Runtime)

1. Webcam Frame Capture: OpenCV captures frames at ~30 fps from the player's camera.

2. Hand Detection & Landmark Extraction: MediaPipe Hands detects both hands and returns 42 (x, y, z) landmark coordinates in normalized image space.
3. Skeleton Rendering: The 42 landmarks are drawn as a stick-figure skeleton (joint dots + bone lines) on a fixed black 128x128 canvas. The skeleton is centred and scale-normalized so the landmark bounding box fills ~80% of the canvas, ensuring distance-invariant CNN input.
4. CNN Inference: The 128x128 grayscale skeleton image is passed through the trained CNN, producing a probability distribution over 14 classes.
5. Stability Confirmation: A sign is confirmed only when the CNN outputs the same class with confidence ≥ 0.85 for 10 consecutive frames (~333 ms at 30 fps). Single-frame noise is absorbed, and the private buffer entry briefly locks in with a visual pulse.
6. Buffer Update: On confirmation, the sign name appends to the private sequence buffer and a brief visual flash appears in the React HUD.

5.4 CNN Architecture

Layer	Type / Config	Output Shape	Notes
Input	RGB Skeleton Image (3 channels)	3 x 128 x 128	Not raw webcam pixels
Conv1	Conv2d(3 -> 32, kernel=3, padding=1) + BatchNorm + ReLU	32 x 128 x 128	
Pool1	MaxPool2d(2x2)	32 x 64 x 64	
Conv2	Conv2d(32 -> 64, kernel=3, padding=1) + BatchNorm + ReLU	64 x 64 x 64	
Pool2	MaxPool2d(2x2)	64 x 32 x 32	
Conv3	Conv2d(64 -> 128, kernel=3, padding=1) + BatchNorm + ReLU	128 x 32 x 32	
GlobalPool	AdaptiveAvgPool2d(4) (Forcing a grid lock)	128 x 4 x 4	
Flatten	-	2,048 parameters (128*4*4)	
FC1	Linear(2048 -> 64) + ReLU + Dropout(0.3)	64	
Output	Linear(32->15) + Softmax	15	11 jutsu + Ram(Act.) + Scrap + Unknown + Shadow clone

Framework: PyTorch. Loss: CrossEntropyLoss. Optimizer: Adam (lr=1e-3). LR Scheduler: StepLR (step_size=10, gamma=0.5). Approximate parameter count: ~1.6M. Expected training time: under 20 minutes on a laptop CPU with the recommended dataset.

5.5 Optional Extension: CNN-LSTM Temporal Model

- The FC1 output (256-dim) for each of the last 15 frames is collected as a temporal sequence
- Fed into a 2-layer LSTM; the final hidden state passes through a linear classification head
- Captures subtle motion between poses, improving accuracy on visually similar signs
- Architecture precedent: CNN-LSTM for video action recognition
- Recommended to document as future work in the project report even if not fully implemented

5.6 Data Augmentation (Applied Offline During Training)

- Random rotation: +/-15 degrees on skeleton images
- Random horizontal flip (simulates angle variation; both hands remain in frame)
- Gaussian noise on raw landmark coordinates before rendering (sigma = 0.005 in normalized space)
- Random brightness jitter on the rendered skeleton: +/-20% pixel intensity
- Random erasing: mask 10% of skeleton pixels to simulate partial occlusion

5.7 In-Game Calibration Screen

Before a match begins, each player runs a local calibration screen in the React UI. This is NOT data collection; it is a live accuracy check using the already-trained model. The calibration screen shows: the player's webcam feed with the MediaPipe landmark overlay rendered in real time, the CNN's current top prediction and confidence percentage updated every frame, and a prompt to hold each sign for 3 seconds to verify the model recognizes it correctly on that player's specific hands and environment. Both players must confirm Ready before the match can start.

6. UI / UX Specification

6.1 Frontend Technology Stack

The game UI runs in a browser tab using React (Vite) rather than OpenCV windows or Pygame. This enables a polished, fully Naruto-themed interface with smooth animations that would be impractical to achieve in a desktop window UI.

Layer	Technology	Purpose
Framework	React 18 + Vite	Component-based UI; fast HMR during development
Styling	Tailwind CSS + custom CSS	Utility classes for layout; custom CSS for Naruto theme (orange/dark palette, glow effects, chakra bar animations)
Fonts	Google Fonts: Cinzel / Noto Sans JP	Cinzel for headings (epic fantasy feel); Noto for body text
Animation	Framer Motion	Action banner entrance/exit animations, HP bar transitions, chakra bar pulse on regen
WebSocket	Native browser WebSocket API	Persistent connection to the Python backend; no extra library needed
Camera	Navigator.mediaDevices (WebRTC)	Access local webcam; feed the local CV pipeline and local preview overlay.
State Management	React useState / useReducer	Local game state; updated via incoming WebSocket game-state messages

6.2 Naruto Theme Design Language

Element	Design Spec
Background	Dark navy (#0D1117) with a subtle repeating kunai/leaf motif at low opacity
Primary accent	Naruto orange (#D4571A) for borders, active elements, and headings
HP bar	Gradient from green (#22C55E) to yellow to red as HP decreases; glows orange at full health
Chakra bar	Deep blue (#1D4ED8) to cyan gradient; pulses with a soft glow animation on turn-start regen
Action banner	Large bold text with a brushstroke-style background overlay; slides in from top on activation
Sign buffer	Horizontal pill tags for each confirmed sign; fades in one at a time as signs are recognized
Turn indicator	Prominent banner strip: 'YOUR TURN' in orange glow / 'OPPONENT'S TURN' in dim grey
Active modifiers	Small buff/debuff badges with countdown.

6.3 Battle Screen Layout

The battle screen is a two-column CSS Grid layout. Player 1 (host) is always left; Player 2 (guest) always right.

UI Zone	Location	Content	Visible To
Camera Feed	Main body of each column	Live webcam stream; host-only toggle can show/hide detection overlay (bounding boxes, landmarks, confidence); remote player video is rendered via WebRTC.	Both players
Action Banner	Absolute overlay at top of caster's column	Animated jutsu/action name; slides in for 2.5s after Activation	Both players
Player Name	Top of each column, above camera	Static label: 'Player 1' / 'Player 2'	Both players
HP Bar	Below camera, each column	Animated gradient bar; HP value shown numerically	Both players
Chakra Bar	Below HP bar, each column	Horizontal pill tags for each sign in the current sequence; each confirmed sign briefly locks in with a pulse animation.	Both players
Sign Buffer	Below player name, own column only	Horizontal pill tags for each sign in the current sequence; e.g. Snake -> Horse	Own player only — never sent to opponent
Active Modifiers	Corner badge area of each column	Colour-coded buff/debuff badges with turn countdown	Both players
Turn Indicator	Full-width strip between the two columns	'YOUR TURN' (orange) or 'OPPONENT'S TURN' (grey)	Both players
Last Action Log	Bottom strip, full width	Scrolling log of last 3 actions, e.g. 'P1: Fireball Jutsu -> 30 dmg'	Both players

6.4 Action Banner Variants

Outcome	Banner Text	Style
Hit	JUTSU NAME!	White text, semi-transparent orange brushstroke background, slides in from top
Miss	JUTSU NAME! - MISSED!	Grey text, dark background, smaller scale
Fail (no chakra)	JUTSU FAILED - Not enough chakra	Red text, shaking entrance animation
Fail (no match)	INCORRECT SEQUENCE (- XX chakra)	Red text, penalty shown inline
Focus	Focus...	Calm blue text, slow fade in, no background box
Shadow Clone	SHADOW CLONE JUTSU!	White on deep blue, lightning glow effect

6.5 Screen Navigation Flow

17. Splash / Lobby: Host and Join buttons. Host sees the ngrok URL to share. Guest pastes URL to connect.
18. Calibration Screen: Live CNN confidence check per sign. Both players click Ready to proceed.
19. Battle Screen: Main gameplay as described in Section 6.3.
20. Match End Screen: Winner display, final HP/chakra values, Rematch button.

7. Networking Architecture

7.1 Technology Stack

Component	Technology	Rationale
Game Logic Server	Python + websockets library	Authoritative game state; all jutsu resolution runs here
CV Inference Server	Python subprocess or same process	CNN inference runs locally on each player's machine; only action intents are sent to the game server
Public Tunnel	Cloudflare Tunnel (or similar reverse tunnel)	Exposes the host's local WebSocket/signaling endpoint on a stable public URL
Frontend Client	React (Vite) in browser	Connects to game server via native WebSocket; renders all UI
Video Streaming	WebRTC media stream	Low-latency remote video path; smoother than base64-over-WebSocket and not used for game logic
Game State Sync	JSON messages	Small, typed event payloads; human-readable for debugging

7.2 Session Establishment

21. Player 1 runs: `python server.py` The game server starts on `localhost:8765` and Cloudflare Tunnel creates a public URL, which is printed to the console and displayed in the React lobby.
22. Player 1 shares the `wss://` URL with Player 2.
23. Player 2 opens the React app and enters the URL in the Join field. Both players see the Lobby screen.
24. Both players complete calibration and click Ready.
25. Player 1 clicks Start Match. The match begins.

7.3 Message Protocol

Message Type	Direction	Key Payload Fields
PLAYER_JOINED	Server -> Both	{ player_id, name }
WEBRTC_SIGNAL	Client -> Server -> Other	{ player_id, sdp, ice_candidate }
SIGN_DETECTED	Client -> Server	{ player_id, sign_name, confidence, buffer }
ACTION_SUBMIT	Client -> Server	{ player_id, sequence, resolved_action_type }

Message Type	Direction	Key Payload Fields
STATE_UPDATE	Server -> Both	{ hp_p1, hp_p2, chakra_p1, chakra_p2, whose_turn, active_modifiers_p1, active_modifiers_p2 }
TURN_RESULT	Server -> Both	{ action_name, hit, dmg_dealt, chakra_delta, status_effects, banner_text, regen_applied }
MATCH_END	Server -> Both	{ winner_id, final_hp_p1, final_hp_p2 }
PING / PONG	Both <-> Both	5s interval heartbeat; disconnect declared after 3 missed PONGs

The sign buffer is never transmitted to the server until the player activates. Only completed action intents are sent. This guarantees the opponent cannot observe the sequence being built. Actual video payloads use the WebRTC media channel; the server only relays signaling and game state.

8. Full Technical Stack

Layer	Technology	Purpose
Frontend Language	JavaScript / JSX	React component logic
Frontend Framework	React 18 + Vite	UI rendering; fast dev server with HMR
Styling	Tailwind CSS + custom CSS	Layout utilities + Naruto-themed design tokens
Animation	Framer Motion	Banner animations, bar transitions, badge pulse effects
Backend Language	Python 3.10+	Game logic server and CV inference pipeline
CV Detection	MediaPipe 0.10+	Two-hand landmark extraction (42 points per frame)
CV Rendering	OpenCV (cv2)	Webcam capture and skeleton image rendering for CNN input
Deep Learning	PyTorch 2.x	CNN training, validation, and live inference
Networking	Python websockets library	WebSocket server handling all clients
Tunnelling	Cloudflare	
Data / Augment	NumPy, torchvision	Dataset management and augmentation pipeline
Config / Registry	YAML config file + registry dict	Balance values, cooldowns, penalties, and jutsu metadata are editable without code changes
Model Storage	PyTorch .pt checkpoint	Exported weights loaded at runtime on each player's machine

9. Development Phases & Milestones

Phase	Deliverable	Key Tasks
Phase 1 Data & Model	Trained CNN $\geq 90\%$ val accuracy	Write data_collector.py -> collect 500+ samples/class across multiple people -> train CNN -> review confusion matrix -> export model.pt
Phase 2 Game Engine	Single-machine offline battle	Turn engine, passive regen, jutsu registry, buff/debuff system, spam dampening, win condition; test with keyboard input
Phase 3 CV Integration	CV-driven single-machine battle	Connect CNN inference to game engine; real-time buffer filling via webcam; calibration screen
Phase 4 Networking	Two-machine online battle via ngrok	WebSocket server and client; video frame streaming; authoritative state sync; pyngrok auto-tunnel
Phase 5 React UI	Full Naruto-themed browser frontend	React + Vite + Tailwind setup; all HUD zones; Framer Motion banners; chakra bar glow animations; calibration screen in browser
Phase 6 Testing & Report	Playtest report + model analysis	End-to-end match tests; latency measurement; CNN accuracy across players; passive regen balance check; LSTM extension documented

10. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
CNN trained on developer's hands does not generalize to friend's hands	High	High	Collect data from at least 2-3 different people during Phase 1; apply all augmentations; the calibration screen reveals issues before the match starts
Visually similar signs confuse the CNN (e.g. Rat vs Ox, Hare vs Dragon)	Medium	High	Review confusion matrix after Phase 1; swap or redesign visually similar sign pairs if per-class accuracy < 85%
Balance values make chakra economy too forgiving	Medium	Medium	Tune jutsu chakra costs and cooldowns in config during Phase 6 playtesting if matches feel too long; reduce passive regen if needed
Video stream too laggy or unstable	Medium	Low	Use WebRTC media with adaptive bitrate; lower resolution/FPS only if bandwidth is a problem
MediaPipe loses hand tracking mid-sequence	Medium	Medium	Require good lighting and full-hand visibility; calibration screen flags unstable hand detection before the match starts
React frontend WebSocket drops on page reload during match	Low	High	Server holds full game state; reconnect message resumes from current state without restarting the match
Scrap sign triggered accidentally	Low	Medium	Crossed-wrists pose is maximally distinct; Scrap uses 15 confirmation frames vs 10 for jutsu signs

Appendix A - Glossary

Term	Definition
Activation Sign (Ram)	The Ram hand sign, reserved as the token that finalizes the sequence buffer and submits it to the game engine. Never used in any jutsu sequence.
Buff	A positive stat modifier applied to the caster for a set number of turns
Chakra	The in-game resource consumed by jutsu; regenerates +10 passively each turn start and an additional +20 on Focus turns; cannot exceed Max Chakra or go below 0
CNN	Convolutional Neural Network; the student-trained image classifier at the core of the CV pipeline
Debuff	A negative stat modifier applied to the opponent for a set number of turns
Focus	The action triggered by activating with an empty buffer; grants an additional +20 chakra on top of the passive +10 turn regen for a total of +30 that turn
Jutsu	Any registered game action triggered by a sign sequence
Jutsu Registry	The dictionary mapping sign-sequence tuples to jutsu stat objects; validated for uniqueness at startup
Landmark	One of 21 keypoints (x, y, z) extracted per hand by MediaPipe; 42 total for two-handed signs
MediaPipe	Google's framework used for hand detection and landmark extraction; not the student's classifier
Miss	A random outcome based on miss_rate that causes a jutsu to have no effect despite a valid cast
Passive Turn Regen	The automatic +10 chakra granted at the start of every player's turn before any action is taken
Scrap Sign	The crossed-wrists reserved pose that clears the sequence buffer without ending the turn
Sequence Buffer	The ordered list of confirmed signs the active player has performed in the current turn, before Activation
Shadow Clone Jutsu	The dodge action: costs 50% of post-regen current chakra; opponent's next attack receives +50% miss rate
Skeleton Image	The 128x128 grayscale rendering of MediaPipe landmarks used as CNN input
Spam Dampening	Damage reduction applied when the same attack jutsu is used on consecutive turns
Unknown Class	The 14th CNN output class that absorbs ambiguous or transitional hand poses between signs; has no in-game action